

Multi-type Populations

EdgeBasedModels.jl Vignette 05

Simon Frost

2026-03-30

Table of contents

Introduction	1
Setup	2
Multivariate PGFs	2
PGF properties	2
Two-type SIR model	3
Building the model	4
Solving the two-type model	6
Extracting population trajectories	8
Assortative mixing with a contact matrix	10
Scaling to more types	14
Summary	17

Introduction

Real populations are rarely homogeneous. Age groups, risk groups, spatial patches, and other forms of heterogeneity lead to structured contact patterns where individuals of different types mix at different rates. Multi-type edge-based compartmental models extend the EBCM framework to track the epidemic on each type of edge separately.

For K population types and M disease stages, the package automatically generates $K^2(1 + M) + K$ ODEs — the K^2 comes from tracking θ_{jl} (probability that an edge from type l to type j has not transmitted) for every ordered pair of types, K^2M from the ϕ variables for each disease stage and type pair, and K from the recovered fraction for each type. This automates what would be extremely tedious bookkeeping by hand.

Setup

```
using EdgeBasedModels
using ModelingToolkit
using Symbolics
using OrdinaryDiffEq
using Plots
```

Multivariate PGFs

In a multi-type network, each type l has its own probability generating function $\psi_l(x_1, \dots, x_K)$ that describes its connections to all K types. For a type- l node, the variable x_k corresponds to edges connecting to type- k neighbors. The PGF encodes the joint degree distribution — the probability that a randomly chosen type- l node has d_1 type-1 neighbors, d_2 type-2 neighbors, etc.

For Poisson degree distributions (the simplest case), the multivariate PGF takes the form:

$$\psi_l(x_1, \dots, x_K) = \exp\left(\sum_{k=1}^K \kappa_{lk}(x_k - 1)\right)$$

where κ_{lk} is the mean number of type- k neighbors that a type- l node has.

Let's define a two-type network with “Young” and “Old” populations:

```
@parameters κ_YY κ_YO κ_OY κ_OO
pgf_Y = multivariate_poisson_pgf([:Young, :Old], Dict{:Young => κ_YY, :Old => κ_YO})
pgf_O = multivariate_poisson_pgf([:Young, :Old], Dict{:Young => κ_OY, :Old => κ_OO})
```

```
MultivariatePGF([:Young, :Old], Any[z_Young, z_Old], exp((-1 + z_Old)*κ_OO + (-1 + z_Young)*κ_OY))
```

Here κ_{YY} is the mean number of Young neighbors a Young individual has, κ_{YO} is the mean number of Old neighbors a Young individual has, and so on.

PGF properties

We can compute mean degrees and partial derivatives symbolically:

```
println("Mean Young neighbors of a Young node: ", mean_degree(pgf_Y, :Young))
println("Mean Old neighbors of a Young node:   ", mean_degree(pgf_Y, :Old))
println("Mean Young neighbors of an Old node:  ", mean_degree(pgf_O, :Young))
println("Mean Old neighbors of an Old node:    ", mean_degree(pgf_O, :Old))
```

Mean Young neighbors of a Young node: κ_{YY}
Mean Old neighbors of a Young node: κ_{YO}
Mean Young neighbors of an Old node: κ_{OY}
Mean Old neighbors of an Old node: κ_{OO}

Partial derivatives describe higher-order degree structure:

```
println("∂ψ_Y/∂x_Young = ", partial_derivative(pgf_Y, :Young, 1))
println("∂²ψ_Y/∂x_Young² = ", partial_derivative(pgf_Y, :Young, 2))
println("∂²ψ_Y/(∂x_Young ∂x_Old) = ", mixed_partial(pgf_Y, :Young, :Old))
```

$\partial\psi_Y/\partial x_{\text{Young}} = \exp((-1 + z_{\text{Old}})\kappa_{YO} + (-1 + z_{\text{Young}})\kappa_{YY})\kappa_{YY}$
 $\partial^2\psi_Y/\partial x_{\text{Young}}^2 = \exp((-1 + z_{\text{Old}})\kappa_{YO} + (-1 + z_{\text{Young}})\kappa_{YY})(\kappa_{YY}^2)$
 $\partial^2\psi_Y/(\partial x_{\text{Young}} \partial x_{\text{Old}}) = \exp((-1 + z_{\text{Old}})\kappa_{YO} + (-1 + z_{\text{Young}})\kappa_{YY})\kappa_{YO}\kappa_{YY}$

We can also evaluate the PGF at specific points:

```
println("ψ_Y(1, 1) = ", eval_multivariate_pgf(pgf_Y, Dict{:Young => 1, :Old => 1}))
```

$\psi_Y(1, 1) = 1$

Two-type SIR model

Now we combine the multivariate PGFs with a disease progression to build a multi-type edge-based model. We'll use a standard SIR progression:

```
@parameters β γ
progression = DiseaseProgression(
    [
        DiseaseStage(:I; transmission_rate = β),
        DiseaseStage(:R; transmission_rate = 0),
    ],
    [DiseaseTransition(:I, :R, γ)];
    entry = :I,
)
```

DiseaseProgression(:S, :I, DiseaseStage[DiseaseStage(:I, β), DiseaseStage(:R, 0)], DiseaseTr

Building the model

The MultiTypeConfigurationModel constructor takes the list of types, one PGF per type, and the disease progression:

```
model = MultiTypeConfigurationModel(
    types = [:Young, :Old],
    pgfs = Dict(:Young => pgf_Y, :Old => pgf_O),
    progression = progression,
)
result = build_edge_system(model)
```

EdgeModelSystem(Model multitype_ebm:

Equations (14):

14 standard: see equations(multitype_ebm)

Unknowns (14): see unknowns(multitype_ebm)

R_Old(t)

R_Young(t)

$\varphi_{R_Old_Old}(t)$

$\varphi_{R_Old_Young}(t)$

?

Parameters (6): see parameters(multitype_ebm)

κ_{YY}

κ_{YO}

κ_{OO}

κ_{OY}

?

Observed (8): see observed(multitype_ebm), Dict{Symbol, Any}(: $\varphi_{I_Old_Old}$ => $\varphi_{I_Old_Old}(t)$,

Let's inspect what the model builder generated:

```
n_eqs = length(ModelingToolkit.equations(result.system))
println("Number of equations: ", n_eqs)
```

Number of equations: 14

With $K = 2$ types and $M = 2$ disease stages (I, R), we expect $K^2(1 + M) + K = 4 \times 3 + 2 = 14$ equations.

```
println("State variables:")
for (name, var) in sort(collect(result.variables), by = x → string(x[1]))
    println("  ", name, " → ", var)
end
```

State variables:

```
R_Old → R_Old(t)
R_Young → R_Young(t)
θ_Old_Old → θ_Old_Old(t)
θ_Old_Young → θ_Old_Young(t)
θ_Young_Old → θ_Young_Old(t)
θ_Young_Young → θ_Young_Young(t)
φ_I_Old_Old → φ_I_Old_Old(t)
φ_I_Old_Young → φ_I_Old_Young(t)
φ_I_Young_Old → φ_I_Young_Old(t)
φ_I_Young_Young → φ_I_Young_Young(t)
φ_R_Old_Old → φ_R_Old_Old(t)
φ_R_Old_Young → φ_R_Old_Young(t)
φ_R_Young_Old → φ_R_Young_Old(t)
φ_R_Young_Young → φ_R_Young_Young(t)
```

```
println("\nObservable quantities:")
for (name, obs) in sort(collect(result.observables), by = x → string(x[1]))
    println("  ", name)
end
```

Observable quantities:

```
I_Old
I_Young
S_Old
S_Young
edge_hazard_Old_Old
edge_hazard_Old_Young
edge_hazard_Young_Old
edge_hazard_Young_Young
excess_hazard_Old_Old
excess_hazard_Old_Young
excess_hazard_Young_Old
excess_hazard_Young_Young
φ_S_Old_Old
```

$\varphi_{S_Old_Young}$
 $\varphi_{S_Young_Old}$
 $\varphi_{S_Young_Young}$

The variables follow a naming convention:

- θ_{jl} (θ_{Young_Young} , θ_{Old_Young} , etc.): probability that an edge from type l to type j has not transmitted infection
- $\phi_{m,jl}$ ($\phi_{I_Young_Young}$, $\phi_{R_Old_Young}$, etc.): probability that a type- j neighbor reached via a type- l edge is in disease stage m
- R_l (R_{Young} , R_{Old}): fraction of type- l population that has recovered

Solving the two-type model

We assign numeric parameter values representing a contact structure where Young individuals have more contacts (especially with each other), and Old individuals have fewer contacts overall:

Parameter	Value	Meaning
κ_{YY}	6	Young $\rightarrow$ Young contacts
κ_{YO}	2	Young $\rightarrow$ Old contacts
κ_{OY}	2	Old $\rightarrow$ Young contacts
κ_{OO}	4	Old $\rightarrow$ Old contacts
β	0.3	Transmission rate
γ	0.1	Recovery rate

```

κ_YY_val = 6.0
κ_YO_val = 2.0
κ_OY_val = 2.0
κ_OO_val = 4.0
β_val = 0.3
γ_val = 0.1
ε = 1e-3
tspan = (0.0, 80.0)

```

(0.0, 80.0)

Set up initial conditions and parameters:

```

ic = default_initial_conditions(result;  $\epsilon = \epsilon$ )
params = Dict(
     $\kappa_{YY} \Rightarrow \kappa_{YY\_val}$ ,  $\kappa_{Y0} \Rightarrow \kappa_{Y0\_val}$ ,
     $\kappa_{0Y} \Rightarrow \kappa_{0Y\_val}$ ,  $\kappa_{00} \Rightarrow \kappa_{00\_val}$ ,
     $\beta \Rightarrow \beta\_val$ ,  $\gamma \Rightarrow \gamma\_val$ ,
)

prob = ODEProblem(result.system, merge(ic, params), tspan)
sol = solve(prob, Tsit5())

```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 13-element Vector{Float64}:

```

0.0
0.13029530140216555
1.2948016038291335
3.7081001889206657
6.978860264954637
11.187330174050185
16.463312041187592
22.915841218826685
30.711003590011646
40.062314334177145
51.297456291845805
64.90215634371162
80.0

```

u: 13-element Vector{Vector{Float64}}:

```

[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0.999, 0.999, 0.999]
[7.743753477668102e-5, 0.00010314696826831724, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,
[0.0007265061906607244, 0.0009677078591270417, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,
[0.0018533838979320056, 0.0024687114673822325, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,
[0.0030051578133951057, 0.004002876880229584, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,
[0.004027741576925763, 0.005364960723843306, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0
[0.004828956460312673, 0.006432180727568163, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0
[0.005377197863543582, 0.007162439494011556, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0
[0.005704615385167032, 0.00759856035982655, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0.
[0.005873100013785586, 0.007822982259257295, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0
[0.005946564796991709, 0.007920837513612356, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0
[0.005972874669177749, 0.007955882321783785, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0
[0.005979979729564275, 0.007965346277995031, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0

```

Extracting population trajectories

For Poisson PGFs, we can compute the susceptible fraction of each type from the θ values. For type l :

$$S_l(t) = \psi(\theta_{1l}(t), \dots, \theta_{Kl}(t)) = \exp\left(\sum_{k=1}^K \kappa_{lk}(\theta_{kl}(t) - 1)\right)$$

```
# Extract  $\theta$  trajectories
 $\theta_{YY}$  = compartment(sol, result, : $\theta_{\text{Young\_Young}}$ )
 $\theta_{OY}$  = compartment(sol, result, : $\theta_{\text{Old\_Young}}$ )
 $\theta_{YO}$  = compartment(sol, result, : $\theta_{\text{Young\_Old}}$ )
 $\theta_{OO}$  = compartment(sol, result, : $\theta_{\text{Old\_Old}}$ )

# Compute S for each type using the Poisson PGF formula
 $S_{\text{Young}}$  = exp.( $\kappa_{YY\_val}$  .* ( $\theta_{YY}$  .- 1) .+  $\kappa_{YO\_val}$  .* ( $\theta_{OY}$  .- 1))
 $S_{\text{Old}}$    = exp.( $\kappa_{OY\_val}$  .* ( $\theta_{YO}$  .- 1) .+  $\kappa_{OO\_val}$  .* ( $\theta_{OO}$  .- 1))

# Extract R for each type
 $R_{\text{Young}}$  = compartment(sol, result, : $R_{\text{Young}}$ )
 $R_{\text{Old}}$    = compartment(sol, result, : $R_{\text{Old}}$ )

#  $I = 1 - S - R$ 
 $I_{\text{Young}}$  = 1.0 .-  $S_{\text{Young}}$  .-  $R_{\text{Young}}$ 
 $I_{\text{Old}}$    = 1.0 .-  $S_{\text{Old}}$  .-  $R_{\text{Old}}$ 
```

13-element Vector{Float64}:

```
0.005982035946064723
0.005904598411288042
0.005255529755403999
0.0041286520481327174
0.0029768781326696176
0.0019542943691389604
0.0011530794857520501
0.0006048380825211409
0.0002774205608976916
0.00010893593227913681
3.547114907301394e-5
9.161276886974004e-6
2.0562165004480576e-6
```



```

plot(sol.t, S_Young, label="S Young", lw=2, color=1)
plot!(sol.t, I_Young, label="I Young", lw=2, color=2)
plot!(sol.t, R_Young, label="R Young", lw=2, color=3)
plot!(sol.t, S_Old, label="S Old", lw=2, ls=:dash, color=1)
plot!(sol.t, I_Old, label="I Old", lw=2, ls=:dash, color=2)
plot!(sol.t, R_Old, label="R Old", lw=2, ls=:dash, color=3)
xlabel!("Time")
ylabel!("Fraction")
title!("Two-type SIR: Young (solid) vs Old (dashed)")

```

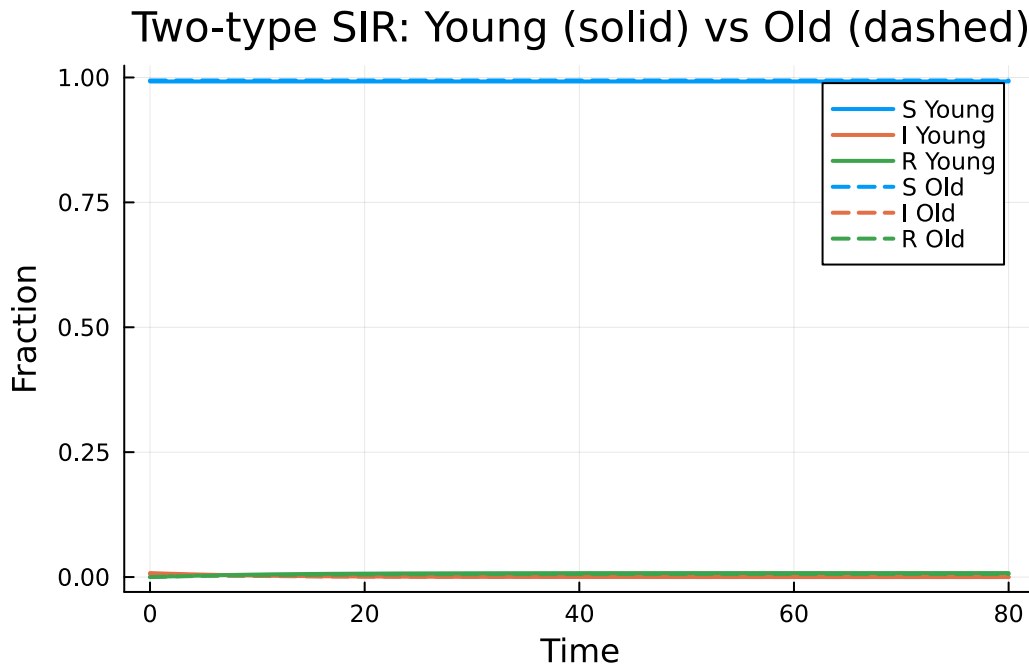


Figure 1: SIR dynamics in a two-type (Young/Old) population. Young individuals have more contacts and experience a faster, larger epidemic.

The Young population (solid lines) experiences a faster epidemic wave because they have more total contacts ($\kappa_{YY} + \kappa_{YO} = 8$) compared to the Old population ($\kappa_{OY} + \kappa_{OO} = 6$, dashed lines).

```

plot(sol.t, I_Young, label="I Young", lw=2, color=1)
plot!(sol.t, I_Old, label="I Old", lw=2, color=2)
xlabel!("Time")
ylabel!("Prevalence")
title!("Infection prevalence by type")

```

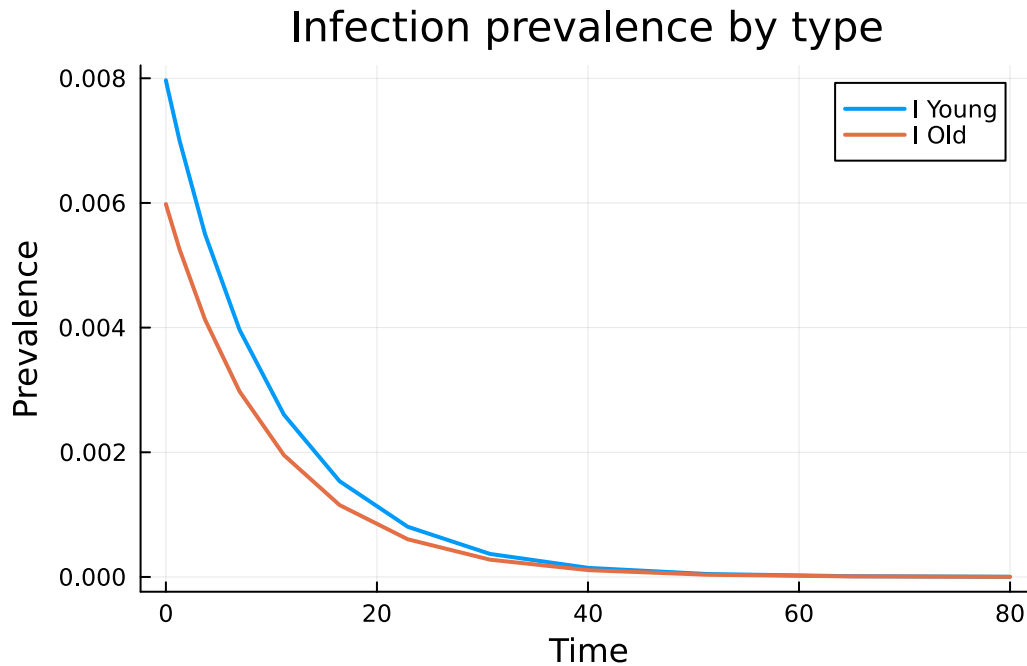


Figure 2: Prevalence (infected fraction) comparison between Young and Old populations.

Assortative mixing with a contact matrix

By default, the multi-type model assumes homogeneous mixing — the transmission rate across an edge depends only on the disease state of the infector. In reality, mixing is often assortative: individuals preferentially contact others of the same type.

The `contact_matrix` parameter scales the transmission rate on each type of edge. A value of 1.0 means the baseline transmission rate applies; values less than 1.0 reduce cross-group transmission:

```
model_assort = MultiTypeConfigurationModel(
    types = [:Young, :Old],
    pgfs = Dict{:Young => pgf_Y, :Old => pgf_O},
    progression = progression,
    contact_matrix = Dict(
        (:Young, :Young) => 1.0,
        (:Old, :Old) => 1.0,
        (:Young, :Old) => 0.5,
        (:Old, :Young) => 0.5,
    ),
)
```

```
)
result_assort = build_edge_system(model_assort)
```

EdgeModelSystem(Model multitype_ebm:

Equations (14):

14 standard: see equations(multitype_ebm)

Unknowns (14): see unknowns(multitype_ebm)

R_Old(t)

R_Young(t)

$\varphi_{R_Old_Old}(t)$

$\varphi_{R_Old_Young}(t)$

?

Parameters (6): see parameters(multitype_ebm)

κ_{YY}

κ_{Y0}

κ_{00}

κ_{0Y}

?

Observed (8): see observed(multitype_ebm), Dict{Symbol, Any}(: $\varphi_{I_Old_Old}$ => $\varphi_{I_Old_Old}(t)$,

The contact matrix entry $c_{jl} = 0.5$ for cross-type edges means that a Young infector transmits to an Old neighbor at half the baseline rate (and vice versa). This models reduced cross-group transmission efficiency due to shorter or less intimate contacts.

```
ic_assort = default_initial_conditions(result_assort;  $\epsilon = \epsilon$ )
prob_assort = ODEProblem(result_assort.system, merge(ic_assort, params), tspan)
sol_assort = solve(prob_assort, Tsit5())
```

retcode: Success

Interpolation: specialized 4th order "free" interpolation

t: 13-element Vector{Float64}:

0.0

0.13029530140216555

1.2948016038291335

3.7081001889206657

6.978860264954637

11.187330174050185

16.463312041187592

22.915841218826685

30.711003590011646

40.062314334177145

51.297456291845805
64.90215634371162
80.0

u: 13-element Vector{Vector{Float64}}:

```
[0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0.999, 0.999, 0.999]  
[7.743753477668102e-5, 0.00010314696826831724, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,  
[0.0007265061906607244, 0.0009677078591270417, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,  
[0.0018533838979320056, 0.0024687114673822325, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,  
[0.0030051578133951057, 0.004002876880229584, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999,  
[0.004027741576925763, 0.005364960723843306, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0  
[0.004828956460312673, 0.006432180727568163, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0  
[0.005377197863543582, 0.007162439494011556, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0  
[0.005704615385167032, 0.00759856035982655, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0.  
[0.005873100013785586, 0.007822982259257295, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0  
[0.005946564796991709, 0.007920837513612356, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0  
[0.005972874669177749, 0.007955882321783785, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0  
[0.005979979729564275, 0.007965346277995031, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.999, 0
```

```
# Extract trajectories for the assortative model  
θ_YY_a = compartment(sol_assort, result_assort, :θ_Young_Young)  
θ_OY_a = compartment(sol_assort, result_assort, :θ_Old_Young)  
θ_YO_a = compartment(sol_assort, result_assort, :θ_Young_Old)  
θ_OO_a = compartment(sol_assort, result_assort, :θ_Old_Old)  
  
S_Young_a = exp.(κ_YY_val .* (θ_YY_a .- 1) .+ κ_YO_val .* (θ_OY_a .- 1))  
S_Old_a = exp.(κ_OY_val .* (θ_YO_a .- 1) .+ κ_OO_val .* (θ_OO_a .- 1))  
R_Young_a = compartment(sol_assort, result_assort, :R_Young)  
R_Old_a = compartment(sol_assort, result_assort, :R_Old)  
I_Young_a = 1.0 .- S_Young_a .- R_Young_a  
I_Old_a = 1.0 .- S_Old_a .- R_Old_a
```

13-element Vector{Float64}:

0.005982035946064723
0.005904598411288042
0.005255529755403999
0.0041286520481327174
0.0029768781326696176
0.0019542943691389604
0.0011530794857520501
0.0006048380825211409
0.0002774205608976916
0.00010893593227913681

```

3.547114907301394e-5
9.161276886974004e-6
2.0562165004480576e-6

```

```

plot(sol.t, I_Young, label="I Young (homogeneous)", lw=2, color=1)
plot!(sol_assort.t, I_Young_a, label="I Young (assortative)", lw=2, ls=:dash, color=1)
plot!(sol.t, I_Old, label="I Old (homogeneous)", lw=2, color=2)
plot!(sol_assort.t, I_Old_a, label="I Old (assortative)", lw=2, ls=:dash, color=2)
xlabel!("Time")
ylabel!("Prevalence")
title!("Homogeneous (solid) vs assortative (dashed) mixing")

```

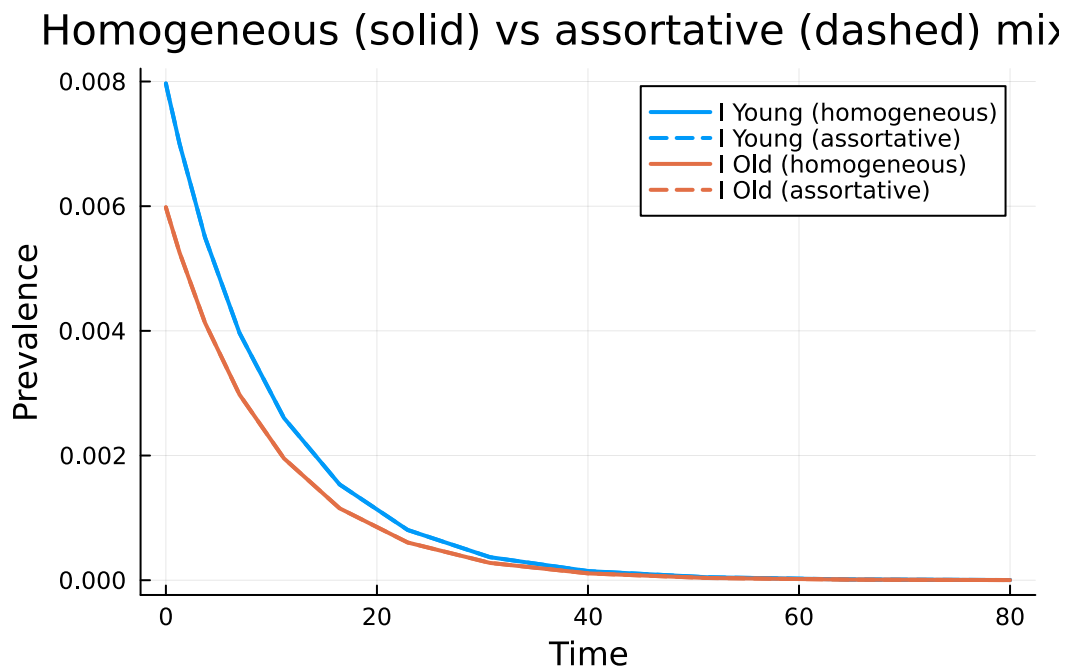


Figure 3: Effect of assortative mixing on epidemic dynamics. Assortative mixing (dashed) slows the epidemic and reduces final size compared to homogeneous mixing (solid).

Assortative mixing tends to slow the epidemic because cross-group transmission is a major driver of epidemic spread — reducing it isolates the two populations, making the overall dynamics more like two separate, smaller epidemics.

```

final_R_Young_hom = R_Young[end]
final_R_Old_hom = R_Old[end]
final_R_Young_assort = R_Young_a[end]

```

```

final_R_Old_assort = R_Old_a[end]

bar_labels = ["Young\n(homogeneous)", "Young\n(assortative)", "Old\n(homogeneous)", "Old\n(assortative)"]
bar_vals = [final_R_Young_hom, final_R_Young_assort, final_R_Old_hom, final_R_Old_assort]
bar(bar_labels, bar_vals, legend=False, ylabel="Final size ( $R_\infty$ )",
    title="Final epidemic size by type and mixing", color=[1 1 2 2],
    ylim=(0, 1))

```

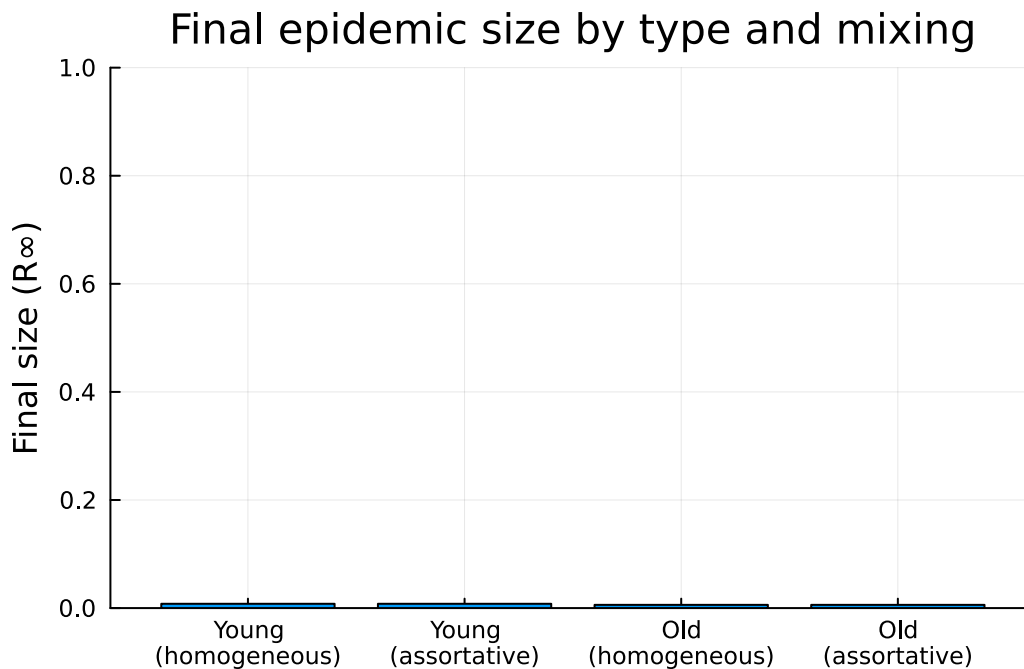


Figure 4: Final epidemic size (recovered fraction) under homogeneous vs assortative mixing.

Scaling to more types

The multi-type framework scales to any number of population types. For example, consider a three-type (Young/Middle/Old) SEIR model:

```

@parameters σ

progression_seir = DiseaseProgression(
    [
        DiseaseStage(:E; transmission_rate = 0),
        DiseaseStage(:I; transmission_rate = β),

```

```

        DiseaseStage(:R; transmission_rate = 0),
    ],
    [
        DiseaseTransition(:E, :I, σ),
        DiseaseTransition(:I, :R, γ),
    ];
    entry = :E,
)

```

DiseaseProgression(:S, :E, DiseaseStage[DiseaseStage(:E, 0), DiseaseStage(:I, β), DiseaseSta

```

@parameters κ_YY3 κ_YM κ_Y03 κ_MY κ_MM κ_M0 κ_OY3 κ_OM κ_O03
types_3 = [:Y, :M, :O]

pgf_Y3 = multivariate_poisson_pgf(types_3, Dict(:Y ⇒ κ_YY3, :M ⇒ κ_YM, :O ⇒ κ_Y03))
pgf_M3 = multivariate_poisson_pgf(types_3, Dict(:Y ⇒ κ_MY, :M ⇒ κ_MM, :O ⇒ κ_M0))
pgf_O3 = multivariate_poisson_pgf(types_3, Dict(:Y ⇒ κ_OY3, :M ⇒ κ_OM, :O ⇒ κ_O03))

model_3type = MultiTypeConfigurationModel(
    types = types_3,
    pgfs = Dict(:Y ⇒ pgf_Y3, :M ⇒ pgf_M3, :O ⇒ pgf_O3),
    progression = progression_seir,
)
result_3type = build_edge_system(model_3type)

n_eqs_3 = length(ModelingToolkit.equations(result_3type.system))
println("Three-type SEIR: $n_eqs_3 equations automatically generated")

```

Three-type SEIR: 39 equations automatically generated

With $K = 3$ types and $M = 3$ stages (E, I, R), we expect $K^2(1 + M) + K = 9 \times 4 + 3 = 39$ equations. Writing these by hand would be impractical and error-prone — the package handles all the symbolic bookkeeping automatically.

```

println("\nState variables (", length(result_3type.variables), " total):")
for (name, _) in sort(collect(result_3type.variables), by = x → string(x[1]))
    println(" ", name)
end

```

State variables (39 total):

R_M
R_O
R_Y
 θ_{M_M}
 θ_{M_O}
 θ_{M_Y}
 θ_{O_M}
 θ_{O_O}
 θ_{O_Y}
 θ_{Y_M}
 θ_{Y_O}
 θ_{Y_Y}
 $\varphi_{E_M_M}$
 $\varphi_{E_M_O}$
 $\varphi_{E_M_Y}$
 $\varphi_{E_O_M}$
 $\varphi_{E_O_O}$
 $\varphi_{E_O_Y}$
 $\varphi_{E_Y_M}$
 $\varphi_{E_Y_O}$
 $\varphi_{E_Y_Y}$
 $\varphi_{I_M_M}$
 $\varphi_{I_M_O}$
 $\varphi_{I_M_Y}$
 $\varphi_{I_O_M}$
 $\varphi_{I_O_O}$
 $\varphi_{I_O_Y}$
 $\varphi_{I_Y_M}$
 $\varphi_{I_Y_O}$
 $\varphi_{I_Y_Y}$
 $\varphi_{R_M_M}$
 $\varphi_{R_M_O}$
 $\varphi_{R_M_Y}$
 $\varphi_{R_O_M}$
 $\varphi_{R_O_O}$
 $\varphi_{R_O_Y}$
 $\varphi_{R_Y_M}$
 $\varphi_{R_Y_O}$
 $\varphi_{R_Y_Y}$

Summary

Multi-type edge-based models capture the essential features of heterogeneous populations:

- Structured contact patterns: Different types can have different degree distributions and mixing preferences.
- Type-specific dynamics: Each population type experiences its own epidemic trajectory, determined by both its own contact structure and cross-group transmission.
- Contact matrices: Assortative or disassortative mixing can be specified to modulate cross-group transmission rates.
- Automatic scaling: The package generates $K^2(1 + M) + K$ equations for any number of types K and disease stages M , handling all the symbolic algebra and bookkeeping automatically.